

Agents with Small Language Models

Optimization Techniques for Reliable Reasoning and Tool Calling

Master Thesis

Paulo Ricardo Beckhauser de Araujo



Agents with Small Language Models

Optimization Techniques for Reliable Reasoning and Tool Calling

Master Thesis

July, 2026

By

Paulo Ricardo Beckhauser de Araujo

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Cover photo: Vibeke Hempler, 2012

Published by: DTU, Department of Applied Mathematics and Computer Science, Richard Petersens Plads, Building 324, 2800 Kgs. Lyngby Denmark
www.compute.dtu.dk

ISSN: [0000-0000] (electronic version)

ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)

ISBN: [000-00-0000-000-0] (printed version)

Approval

This Master's thesis was prepared for the fulfillment for the degree Master of Science in Engineering, MSc Autonomous Systems at the Technical University of Denmark (DTU). The thesis was prepared by Paulo Ricardo Beckhauser de Araujo (student ID: s242779) under supervision of Nicki Skafte Detlefsen, Professor at DTU Compute. The project period lasted 6 months from January 5th, 2026 until July 5th, 2026, equal to a workload of 35 ECTS points.

The reader is expected to be familiar with the basic concepts of linear algebra, programming languages, deep learning and generative AI.

Paulo Ricardo Beckhauser de Araujo - s242779

.....
Signature

.....
Date

Abstract

Large language models have become the default backbone for agentic systems, yet their computational cost and reliance on external APIs limit deployment in latency-sensitive or resource-constrained settings. This thesis investigates whether small language models (1B–8B parameters) can achieve reliable agentic behavior, specifically in reasoning and tool calling, when combined with targeted optimization techniques.

Acknowledgements

This thesis was completed with the guidance and support of **Nicki Skafte Detlefsen**, Associate Professor at the Technical University of Denmark, who supervised this work.

I would also like to thank the professors and friends who were part of my journey throughout my master's degree, and above all, my family for their unwavering support along the way.

Contents

Preface	ii
Abstract	iii
Acknowledgements	iv
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Research Question	1
1.4 Contributions	2
1.5 Thesis Structure	2
2 Background	3
2.1 Language Models	3
2.2 Agentic Systems	4
2.3 Constrained Decoding	4
2.4 Prompt Engineering and Inference-Time Compute	5
2.5 Retrieval-Augmented Generation	5
2.6 Parameter-Efficient Fine-Tuning	6
2.7 Knowledge Distillation	6
2.8 Related Work	7
3 Methodology	8
3.1 Experimental Design	8
3.2 Models	8
3.3 Baseline Definition	8
3.4 Constrained Decoding	8
3.5 Prompt Engineering and Few-Shot	8
3.6 Inference-Time Compute	8
3.7 Retrieval-Augmented Generation	8
3.8 LoRA Fine-Tuning	8
3.9 Knowledge Distillation	8
3.10 Evaluation Framework	8
4 Results	9
4.1 Baseline Performance	9
4.2 No-Training Methods	9
4.3 Training-Based Methods	9
4.4 Combined Configurations	9
4.5 Comparison with Frontier Models	9
5 Discussion	10
5.1 Impact of Constrained Decoding vs. Model Scale	10
5.2 Cost-Benefit Analysis of Each Method	10
5.3 Limitations	10
5.4 Practical Implications	10

6 Conclusion	11
6.1 Answering the Research Question	11
6.2 Future Work	11
Bibliography	12
A Generative AI Disclosure	15
B Hyperparameters and Training Details	16
C Full Benchmark Results	17

1 Introduction

1.1 Motivation

Large language models (LLMs) have become the dominant backbone for agentic systems, powering applications that reason over user requests, decompose tasks, and call external tools to produce structured outputs. Frontier models such as GPT-4 and Claude demonstrate strong performance on these tasks, but their deployment comes with significant trade-offs: high inference costs, dependency on external APIs, increased latency, and limited control over data privacy. For many real-world applications, these constraints make frontier models impractical.

Small language models (SLMs), typically ranging from 1B to 8B parameters, offer an alternative. Recent open-weight models such as Qwen 2.5 [16], Llama 3.2 [5], and Phi-4 [1] can run on consumer-grade hardware, enable local deployment, and drastically reduce per-query costs. However, out of the box, SLMs struggle with the core capabilities that agentic systems demand. They frequently produce malformed structured outputs, select incorrect tools, hallucinate function arguments, and fail at multi-step reasoning. These shortcomings have led to a widespread assumption that reliable agentic behavior requires large-scale models.

This thesis challenges that assumption. Rather than treating model scale as the primary driver of agent reliability, it investigates whether targeted optimization techniques can bridge the performance gap between small and large models for agentic tasks. If small models can be made reliable enough for tool calling and reasoning through methods such as constrained decoding, prompt engineering, and fine-tuning, they could enable a new class of local, cost-efficient, and privacy-preserving agentic systems.

1.2 Problem Statement

Agentic systems built on language models must perform two fundamental capabilities well: reasoning correctly about user requests and calling the right tools with valid arguments. Reasoning requires the model to decompose tasks, decide when to call a tool versus respond directly, and chain multi-step logic without hallucinating intermediate steps. Tool calling requires selecting the correct function from a schema, producing valid structured arguments, and handling the result appropriately.

When small language models are used for these tasks without any optimization, they exhibit several failure modes. They generate invalid JSON, select non-existent tools, produce arguments with incorrect types or values, and fail to follow execution plans. These failures are not merely inconvenient; they render the agent non-functional for production use.

The central question is whether these failures stem from fundamental limitations in model capacity or from insufficient constraints and guidance during generation. If the latter, then a carefully chosen stack of optimization techniques should be able to make small models competitive with frontier models on agentic benchmarks.

1.3 Research Question

This thesis addresses the following research question:

Which combination of optimization techniques enables a small language model to reason effectively and call the correct tools?

To answer this question, the thesis evaluates a cumulative pipeline of methods, ordered by implementation cost. No-training methods are evaluated first (constrained decoding [20], prompt engineering [3], inference-time compute [19], retrieval-augmented generation [12]), followed by training-based methods (LoRA fine-tuning [8], knowledge distillation [6]). Each method is assessed both individually and in combination, measuring its marginal contribution to tool-calling accuracy and reasoning quality.

1.4 Contributions

The main contributions of this thesis are:

1. A systematic evaluation of optimization techniques for small language models in agentic tasks, ordered from no-training to training-required methods.
2. An empirical analysis of constrained decoding as a foundation for reliable structured output generation in small models.
3. A cumulative evaluation framework that measures the marginal impact of each technique on tool-calling correctness and reasoning quality.
4. A comparison of optimized small language model configurations against frontier models on the Berkeley Function Calling Leaderboard (BFCL) [22] and custom benchmarks.
5. Practical guidelines for deploying agentic systems with small models on consumer hardware.

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

Chapter 2 provides the theoretical background on language models, agentic systems, and the optimization techniques investigated in this work, including constrained decoding, prompt engineering, retrieval-augmented generation, LoRA fine-tuning, and knowledge distillation.

Chapter 3 describes the experimental design, the cumulative evaluation ladder, the candidate models, and the evaluation framework including benchmarks and metrics.

Chapter 4 presents the experimental results for each optimization method, both individually and in combination, along with comparisons to frontier models.

Chapter 5 discusses the implications of the results, analyzes the cost-benefit trade-offs of each method, and addresses limitations of the study.

Chapter 6 answers the research question, summarizes the findings, and outlines directions for future work.

2 Background

This chapter introduces the theoretical foundations underlying this thesis. It covers language model architectures and their scaling properties, the design of agentic systems, and the optimization techniques evaluated in subsequent chapters.

2.1 Language Models

2.1.1 Transformer Architecture

The Transformer architecture, introduced by Vaswani et al. [18], forms the foundation of modern language models. It replaces recurrent computation with a self-attention mechanism that allows each token in a sequence to attend to all other tokens in parallel. A Transformer block consists of a multi-head self-attention layer followed by a position-wise feed-forward network, with layer normalization and residual connections applied around each sub-layer.

Given an input sequence of token embeddings X , the self-attention mechanism computes queries $Q = XW^Q$, keys $K = XW^K$, and values $V = XW^V$ through learned linear projections. The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (2.1)$$

where d_k is the dimension of the key vectors. Multi-head attention runs this operation in parallel across multiple subspaces, allowing the model to capture different types of dependencies simultaneously.

For language modeling, the decoder-only variant is most common. It applies a causal attention mask so that each token can only attend to preceding tokens, enabling autoregressive generation where the model predicts one token at a time conditioned on all previous tokens.

2.1.2 Scaling Laws

Scaling laws describe the empirical relationship between model performance and three key factors: the number of parameters, the amount of training data, and the available compute budget. Kaplan et al. [10] showed that language model loss follows a power-law relationship with each of these factors, and that performance improves predictably as any of them increases.

Hoffmann et al. [7] refined these findings with the Chinchilla scaling laws, demonstrating that many large models were under-trained relative to their size. Their work showed that for a fixed compute budget, optimal performance is achieved by balancing model size and training data rather than maximizing parameters alone. This insight has direct implications for small language models: a well-trained small model can outperform a larger but under-trained one.

More recently, the focus has shifted toward inference-time scaling [17], where additional computation at generation time (through techniques such as chain-of-thought prompting or search) can improve performance without increasing model size.

2.1.3 Small vs. Large Language Models

Large language models (LLMs) such as GPT-4 [13] and the Claude model family operate at hundreds of billions of parameters and demonstrate strong performance across a wide range of tasks, including complex reasoning and tool use. However, their size demands

significant computational resources, typically requiring multi-GPU clusters or cloud API access for inference.

Small language models (SLMs), in the range of 1B to 8B parameters, can run on consumer hardware such as a single GPU with 24GB of VRAM. Recent open-weight SLMs, including Qwen 2.5 [16], Llama 3.2 [5], and Phi-4 [1], have narrowed the gap with larger models on standard benchmarks through improved training data, better tokenizers, and architectural refinements such as grouped-query attention (GQA).

Despite these improvements, SLMs still lag behind frontier models on tasks that require multi-step reasoning, tool selection from large schemas, and reliable structured output generation. This gap motivates the optimization techniques explored in this thesis.

2.2 Agentic Systems

An agentic system is a software architecture in which a language model acts as the central reasoning engine, deciding which actions to take, invoking external tools, and synthesizing results to fulfill user requests. Unlike simple question-answering, agentic behavior involves planning, acting, and adapting based on intermediate results.

2.2.1 Tool Calling

Tool calling refers to the ability of a language model to invoke external functions or APIs as part of its response. The model receives a set of available tool schemas (names, descriptions, parameter types) and must select the appropriate tool for a given request, generate valid arguments matching the schema, and incorporate the tool’s response into its reasoning.

Tool calling is the primary capability evaluated in this thesis. It requires both language understanding (to map user intent to the correct tool) and structured generation (to produce syntactically and semantically valid function calls).

2.2.2 Structured Output Generation

Agentic systems typically require the model to produce outputs in a specific format, most commonly JSON. The model must generate output that is syntactically valid (parseable JSON), conforms to a predefined schema (correct field names and types), and is semantically correct (values that make sense for the given context).

Without explicit constraints, language models frequently produce malformed outputs: missing closing brackets, incorrect field names, wrong data types, or extra fields not present in the schema. These failures are particularly common in small models and represent a key reliability bottleneck for agentic deployments.

2.2.3 The ReAct Pattern

ReAct (Reasoning and Acting) [23] is a prompting framework that interleaves reasoning traces with action steps. Rather than generating a final answer directly, the model follows a loop of Thought (reasoning about what to do next), Action (calling a tool or performing a step), and Observation (processing the result of that action).

This pattern improves multi-step task completion by making the model’s reasoning explicit and allowing it to adapt its plan based on intermediate results. ReAct is particularly relevant for agentic systems because it provides a structured way to combine reasoning and tool use within a single generation process.

2.3 Constrained Decoding

Constrained decoding modifies the generation process so that the model can only produce outputs that conform to a predefined structure. Rather than relying on the model to

implicitly learn output formats, constrained decoding enforces them at inference time by masking invalid tokens at each generation step.

2.3.1 Formal Grammars for Generation

The key idea is to define the valid output space using a formal grammar, typically a context-free grammar (CFG) or a JSON schema. At each decoding step, the grammar determines which tokens are valid continuations given the tokens generated so far. Invalid tokens receive a probability of zero (or negative infinity in log-space), ensuring that only grammatically valid sequences can be produced.

For tool calling, this means the model is guaranteed to produce valid JSON that conforms to the tool’s argument schema. Field names, types, and structure are enforced by construction rather than learned behavior. Willard and Louf [20] formalized this approach using finite-state machines compiled from regular expressions and JSON schemas.

2.3.2 Guided Decoding Techniques

Several libraries implement constrained decoding for language models. Outlines [21] uses finite-state machines compiled from regular expressions or JSON schemas to guide generation. LMQL [2] combines constrained decoding with a query language that allows expressing complex output constraints declaratively.

These techniques are particularly impactful for small models because they eliminate an entire class of errors (malformed outputs) without requiring any additional training. Constrained decoding is the foundational method in this thesis and serves as the base layer upon which other techniques are stacked.

2.4 Prompt Engineering and Inference-Time Compute

2.4.1 Few-Shot Prompting

Few-shot prompting provides the model with examples of correct input-output pairs directly in the prompt [3]. For tool calling, this means including examples of user requests paired with the expected function calls. The model uses these examples as templates to guide its own generation.

Few-shot prompting is a zero-cost technique (no training required) that can significantly improve task performance, especially for smaller models that benefit from explicit demonstrations of the expected output format and reasoning process.

2.4.2 Chain-of-Thought Reasoning

Chain-of-thought (CoT) prompting [19] encourages the model to generate intermediate reasoning steps before producing a final answer. Instead of directly outputting a tool call, the model first explains its reasoning: which tool is appropriate, what arguments are needed, and why.

CoT is a form of inference-time compute, trading additional generated tokens for improved accuracy. It is particularly effective for tasks that require multi-step reasoning, such as selecting among many similar tools or constructing complex function arguments. For small models, CoT can compensate for limited implicit reasoning capacity by making the reasoning process explicit.

2.5 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [12] augments the model’s input with relevant information retrieved from an external knowledge source at inference time. In the context

of tool calling, RAG retrieves relevant tool schemas and documentation based on the user’s request, rather than including all available tools in every prompt.

This approach offers two benefits for small models. First, it reduces the prompt length by only including relevant tools, which is important given the limited context windows of smaller models. Second, it reduces hallucination by grounding the model’s generation in retrieved factual content rather than relying solely on parametric knowledge.

A typical RAG pipeline for tool calling involves encoding tool descriptions into a vector store, retrieving the top-k most relevant tools for a given query using semantic similarity, and injecting the retrieved schemas into the model’s prompt before generation.

2.6 Parameter-Efficient Fine-Tuning

Fine-tuning adapts a pre-trained model to a specific task by continuing training on task-specific data. Full fine-tuning updates all model parameters, which is computationally expensive and requires significant memory. Parameter-efficient fine-tuning (PEFT) methods address this by updating only a small subset of parameters while keeping the rest frozen.

2.6.1 LoRA

Low-Rank Adaptation (LoRA) [8] is the most widely used PEFT method. Instead of updating the full weight matrices during fine-tuning, LoRA injects trainable low-rank decomposition matrices alongside the frozen pre-trained weights. For a weight matrix $W \in \mathbb{R}^{d \times k}$, LoRA learns two smaller matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$, where $r \ll \min(d, k)$, such that the adapted weight becomes $W + AB$.

This reduces the number of trainable parameters by orders of magnitude while achieving performance comparable to full fine-tuning on many tasks. For tool calling, LoRA can be used to fine-tune small models on datasets of function-calling examples, teaching them to select tools and generate arguments more accurately.

2.6.2 PEFT Variants

Beyond LoRA, several other PEFT methods exist. QLoRA [4] combines LoRA with quantization, applying low-rank adaptation on top of a 4-bit quantized base model to further reduce memory requirements. Adapter layers insert small trainable modules between frozen Transformer layers. Prefix tuning prepends trainable continuous vectors to the key and value sequences in each attention layer.

This thesis primarily uses LoRA due to its simplicity, strong empirical performance, and wide support across frameworks and model architectures.

2.7 Knowledge Distillation

Knowledge distillation [6] transfers knowledge from a larger teacher model to a smaller student model. The student is trained not only on ground-truth labels but also on the teacher’s output distributions or generated responses. The idea is that the teacher’s outputs contain richer information than hard labels alone, including relative confidence across alternatives and implicit reasoning patterns.

For tool calling, knowledge distillation involves generating a dataset of tool-calling examples using a frontier model (the teacher), then fine-tuning the small model (the student) on these examples. The teacher provides high-quality demonstrations of tool selection, argument construction, and reasoning traces that the student can learn to replicate.

This approach is particularly relevant when labeled tool-calling data is scarce, as the teacher model can generate large volumes of training data from unlabeled user requests.

The trade-off is the cost of running the teacher model to generate the distillation dataset.

2.8 Related Work

Several lines of research are directly relevant to this thesis.

In the domain of function calling and tool use, the Berkeley Function Calling Leaderboard (BFCL) [22] provides a standardized benchmark for evaluating language models on function-calling tasks, measuring AST accuracy and execution accuracy across multiple categories. ToolLLM [15] provides a large-scale benchmark with over 16,000 real-world APIs for evaluating tool-use capabilities. Gorilla [14] demonstrated that fine-tuned smaller models can outperform larger models on API calling when given appropriate training data.

For constrained decoding, Willard and Louf [20] formalized the use of finite-state machines for structured generation in language models. The Outlines library [21] implements this approach for JSON schema-guided generation. LMQL [2] introduced a query language for constrained generation that supports complex output specifications.

In the area of small model optimization, recent work on Phi [1], Qwen [16], and Llama [5] model families has demonstrated that careful data curation and training recipes can produce small models with strong general capabilities. However, systematic evaluations of these models specifically for agentic tasks remain limited.

The intersection of constrained decoding with fine-tuning for tool calling in small models is relatively unexplored. Most existing work applies these techniques in isolation or focuses on large models. This thesis contributes a systematic evaluation of their combined effect on small models for agentic tasks.

3 Methodology

3.1 Experimental Design

3.1.1 Cumulative Evaluation Ladder

3.1.2 Hardware and Infrastructure

3.2 Models

3.2.1 Qwen 2.5

3.2.2 Llama 3.2

3.2.3 Phi-4 Mini

3.2.4 Other Candidates

3.3 Baseline Definition

3.4 Constrained Decoding

3.5 Prompt Engineering and Few-Shot

3.6 Inference-Time Compute

3.6.1 Chain-of-Thought

3.6.2 ReAct

3.7 Retrieval-Augmented Generation

3.8 LoRA Fine-Tuning

3.9 Knowledge Distillation

3.10 Evaluation Framework

3.10.1 Berkeley Function Calling Leaderboard

3.10.2 Custom Tool-Calling Benchmark

3.10.3 Metrics

4 Results

4.1 Baseline Performance

4.2 No-Training Methods

4.2.1 Constrained Decoding

4.2.2 Prompt Engineering and Few-Shot

4.2.3 Inference-Time Compute

4.2.4 Retrieval-Augmented Generation

4.3 Training-Based Methods

4.3.1 LoRA Fine-Tuning

4.3.2 Knowledge Distillation

4.4 Combined Configurations

4.5 Comparison with Frontier Models

5 Discussion

- 5.1 Impact of Constrained Decoding vs. Model Scale
- 5.2 Cost-Benefit Analysis of Each Method
- 5.3 Limitations
- 5.4 Practical Implications

6 Conclusion

6.1 Answering the Research Question

6.2 Future Work

Bibliography

- [1] Marah Abdin et al. “Phi-4 Technical Report”. In: *arXiv preprint arXiv:2412.08905* (2024).
- [2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. “Prompting Is Programming: A Query Language for Large Language Models”. In: *Proceedings of the ACM on Programming Languages* 7.PLDI (2023).
- [3] Tom Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* 33 (2020).
- [4] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized Language Models”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [5] Aaron Grattafiori et al. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024).
- [6] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the Knowledge in a Neural Network”. In: *arXiv preprint arXiv:1503.02531* (2015).
- [7] Jordan Hoffmann et al. “Training Compute-Optimal Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.
- [8] Edward J Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *arXiv preprint arXiv:2106.09685* (2022).
- [9] International Committee of Medical Journal Editors (ICMJE) et al. *Recommendations for the Conduct, Reporting, Editing, and Publication of Scholarly Work in Medical Journals*. <https://www.icmje.org/icmje-recommendations.pdf>. Originally prepared by Severinsen and Ekern (2017), updated by Torp (2022). Translated by Carole Hognestad. Aug. 2020. URL: <https://www.icmje.org/icmje-recommendations.pdf>.
- [10] Jared Kaplan et al. “Scaling Laws for Neural Language Models”. In: *arXiv preprint arXiv:2001.08361* (2020).
- [11] Philipp Lehman et al. *Biblatex – Sophisticated Bibliographies in LaTeX*. 2018. URL: <https://www.ctan.org/pkg/biblatex>.
- [12] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems* 33 (2020).
- [13] OpenAI. “GPT-4 Technical Report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [14] Shishir G Patil et al. “Gorilla: Large Language Model Connected with Massive APIs”. In: *arXiv preprint arXiv:2305.15334* (2023).
- [15] Yujia Qin et al. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs”. In: *arXiv preprint arXiv:2307.16789* (2024).
- [16] Qwen Team. “Qwen2.5 Technical Report”. In: *arXiv preprint arXiv:2412.15115* (2024).
- [17] Charlie Snell et al. “Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters”. In: *arXiv preprint arXiv:2408.03314* (2024).
- [18] Ashish Vaswani et al. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [19] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems* 35 (2022).
- [20] Brandon T Willard and Rémi Louf. “Efficient Guided Generation for Large Language Models”. In: *arXiv preprint arXiv:2307.09702* (2023).
- [21] Brandon T Willard and Rémi Louf. *Outlines: Structured Text Generation*. 2024. URL: <https://github.com/dottxt-ai/outlines>.

- [22] Fanjia Yan et al. *Berkeley Function Calling Leaderboard*. 2024. URL: <https://gorilla.cs.berkeley.edu/leaderboard.html>.
- [23] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *arXiv preprint arXiv:2210.03629* (2023).

A Generative AI Disclosure

B Hyperparameters and Training Details

C Full Benchmark Results

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk